

Walkthrough:

Production_Work

(1).ipynb

This document provides a detailed, step-by-step explanation of the core Jupyter Notebook (`Production_Work (1).ipynb`) that orchestrates the entire AI pipeline, from raw audio data processing to model deployment packaging.

Part 1: Data Preparation and Preprocessing

The first massive chunk of the notebook is dedicated to assembling and meticulously cleaning the training data to ensure it's production-ready.

Step 1: Load and Prepare Datasets

- **Kaggle API Integration:** The notebook programmatically downloads three distinct datasets using Kaggle and Github APIs:
 - `infant_cry_corpus` (used for fine-grained Stage 2 classification).
 - `baby_crying` (used for general cry detection).
 - `baby_sounds` (mixed sounds including non-cry vocalizations).
- **Initial Dataframe:** It consolidates metadata (file paths, labels, source datasets) from all these sources into a massive single pandas dataframe.

Step 2: Standardize Labels

- **Taxonomy Mapping:** The notebook merges acoustically similar, highly-granular labels into **4 clinically actionable categories** to increase class representation and model robustness:
 - `hungry`, `tired`, `discomfort`, `cold/hot` are all mapped to → **needs**
 - `belly_pain` mapped to → **physical_pain**
 - `scared` and `burping` are kept as they are.
- **Dropping Outliers:** It intentionally drops the `lonely` label, likely because it lacked sufficient data or was acoustically indistinguishable from general "needs".

Step 3 & 4: Train/Test Splitting & Adding ESC-50

- **Stratified Split:** The dataset is split into an 80% `pool` (for training/validation) and a 20% `test` (held-out) set. It uses `stratify` on the labels to ensure rare classes (like `burping`) are equally represented in both splits.
- **ESC-50 (Negative Data):** To train the Stage 1 binary classifier to know what *isn't* a baby cry, the ESC-50 dataset (environmental sounds) is downloaded.
 - **Crucial Exclusion:** The author specifically filters out ESC-50 categories that sound like human vocalizations (`crying_baby`, `laughing`, `sneezing`, `breathing`, `coughing`, `snoring`). If these were included as "not_cry", it would confuse the model.

Step 5: Audio Standardization (HuBERT-Ready)

This is where the heavy audio processing happens to normalize everything before it hits the neural network:

- **Resampling:** `librosa` converts everything to mono, 16,000 Hz.
- **High-Pass Filter:** A 50 Hz Butterworth filter removes low-end rumble/DC offset.
- **Loudness Normalization:** `pyloudnorm` sets all audio to a broadcast standard of `-23.0 LUFS`.
- **VAD Trimming:** Uses `webrtcvad` (WebRTC Voice Activity Detection) to intelligently trim pure silence from the beginning and end of recordings.

Step 6: Remove Bad Data

- The script sweeps through the processed files, dropping any that failed the audio pipeline, were excessively short (<0.5 seconds), or were exact duplicates.

Part 2: Model Architecture and PyTorch Lightning

DataModules and Datasets

- **BabyCryDataset:** A custom PyTorch `Dataset` that reads the audio, pads/crops it to exactly 5 seconds (80,000 samples), and generates the binary `attention_mask` required by Transformers.
- **BabyCryDataModule:** A PyTorch Lightning module that handles batching, shuffling, and worker management for `train_dataloader` and `val_dataloader`.

HubertClassifier Module

- The notebook defines the core neural network architecture.
- **Backbone:** Uses `facebook/hubert-base-ls960` from Hugging Face.
- **Freezing:** The feature extractor CNN layers of HuBERT are frozen to save VRAM and prevent catastrophic forgetting.
- **Classification Head:** A custom head is appended: `Linear -> GELU -> Dropout -> Linear`.

- **Masked Mean Pooling:** Before the classification head, the code pools the transformer tokens over time, being careful to only average tokens that aren't masked (ignoring padded silence).
-

Part 3: Training Stages and Experiments

Stage 1: Binary Classification Training

- The model is trained to differentiate `baby_cry` vs. `not_baby_cry`.
- It uses class weights in the CrossEntropy loss to handle the imbalance between the two classes.

Stage 2: 4-Class Classification Training

- The notebook spins up a fresh `HubertClassifier` configured for 4 outputs.
- Heavy inverse-frequency class weighting is applied (e.g., `burping` gets a weight of ~2.07 while `needs` gets a tiny weight of ~0.20) to force the model to care about the minority classes.

The "T4" Experiments: Ensembling and Thresholding

- **Diagnostic Analysis:** The author generates confusion matrices and notices specific failure modes.
 - **Label Smoothing:** Re-trains the model with label smoothing (0.1) to prevent the model from becoming overconfident.
 - **Ensemble Averaging:** Instead of relying on a single checkpoint, the notebook loads *both* the Epoch 6 and Epoch 7 checkpoints, running them simultaneously and averaging their softmax probabilities. This yielded a "free win" in accuracy.
 - **Confidence Thresholds:** Implemented logic to map model confidence to human-readable labels ("High Confidence", "Low Confidence", etc.).
-

Part 4: Production Packaging

The notebook does not just train a model; it acts as an automated release build system.

Build Local-Ready Backend Package

- The notebook creates a `backend_package_v1` directory on Google Drive.
- **Copying Checkpoints:** It copies the best `.ckpt` files (Stage 1 best, and Stage 2 epochs 6 & 7) into a `models/` directory.
- **Generating Configs:** It dumps the Label Encoders (mapping integers to string labels) and training hyperparameters into JSON files inside a `config/` directory.
- **Code Generation:** In a massive string block, the notebook literally writes the `inference/inference.py` script out to the drive, ensuring the PyTorch code required to run the model exactly matches the code used to train it.

- **Requirements:** It generates a `requirements.txt` locking down the exact library versions used during the Colab session.

(This packaged directory is exactly what was copied over to the local repository `c:\BabyCry` that the FastAPI server now runs on.)